



Árboles de Decisión

Un recorrido profundo por uno de los algoritmos más intuitivos e interpretables del aprendizaje automático: desde la idea fundamental hasta sus ventajas, límites y aplicaciones prácticas.

TEMA 8.1

MACHINE LEARNING

Contenido del tema

Exploraremos los fundamentos teóricos y prácticos de los árboles de decisión, siguiendo esta estructura:

01

La idea del árbol

Cómo funciona un árbol de decisión como modelo de clasificación o regresión.

03

Profundidad y sobreajuste

El peligro de crecer demasiado y cómo controlarlo.

02

Criterios de división

Gini e impureza de entropía para elegir el mejor atributo en cada nodo.

04

Poda, interpretabilidad y variantes

Poda, visualización, árboles de regresión, ventajas y límites.



La idea del árbol

Un árbol de decisión imita el proceso de razonamiento humano: **se formulan preguntas sucesivas sobre los atributos de los datos** hasta llegar a una conclusión. Cada nodo interno representa una pregunta (p. ej., «¿edad > 30?»), cada rama es una respuesta posible y cada hoja contiene la predicción final.

Nodo raíz

Primera pregunta, sobre el atributo más informativo del conjunto de datos.

Nodos internos

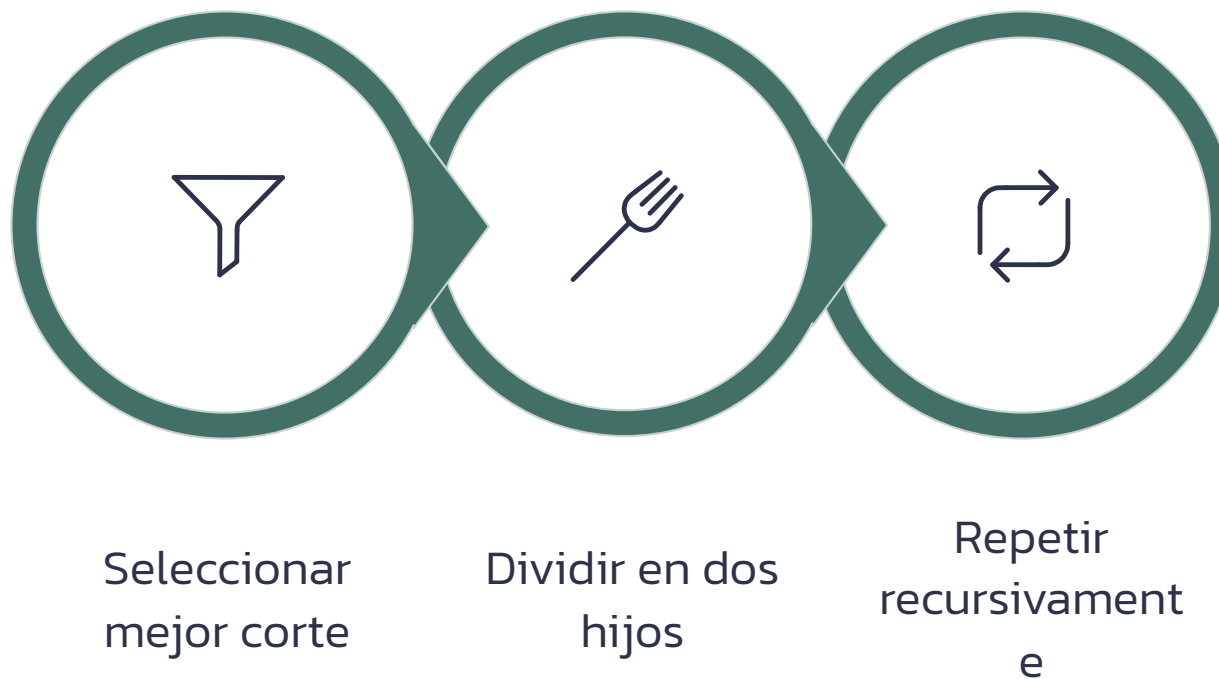
Preguntas sucesivas que van particionando el espacio de características.

Hojas

Decisión final: clase mayoritaria (clasificación) o valor medio (regresión).

¿Cómo aprende un árbol?

El algoritmo de entrenamiento más habitual es **CART** (*Classification and Regression Trees*). En cada nodo busca la partición binaria que maximiza la **pureza** de los subconjuntos resultantes. El proceso es codicioso (*greedy*): toma la mejor decisión local sin garantizar el óptimo global.



El proceso se repite recursivamente en cada nodo hijo hasta que se alcanza un criterio de parada: profundidad máxima, mínimo de muestras por hoja o pureza perfecta.

Criterios de división: Gini y Entropía

Para elegir **qué atributo usar** en cada nodo y con qué umbral, se necesita medir la *impureza* de un conjunto. Los dos criterios más utilizados son el **índice de Gini** y la **entropía de Shannon**.

Índice de Gini

$$Gini = 1 - \sum_k p_k^2$$

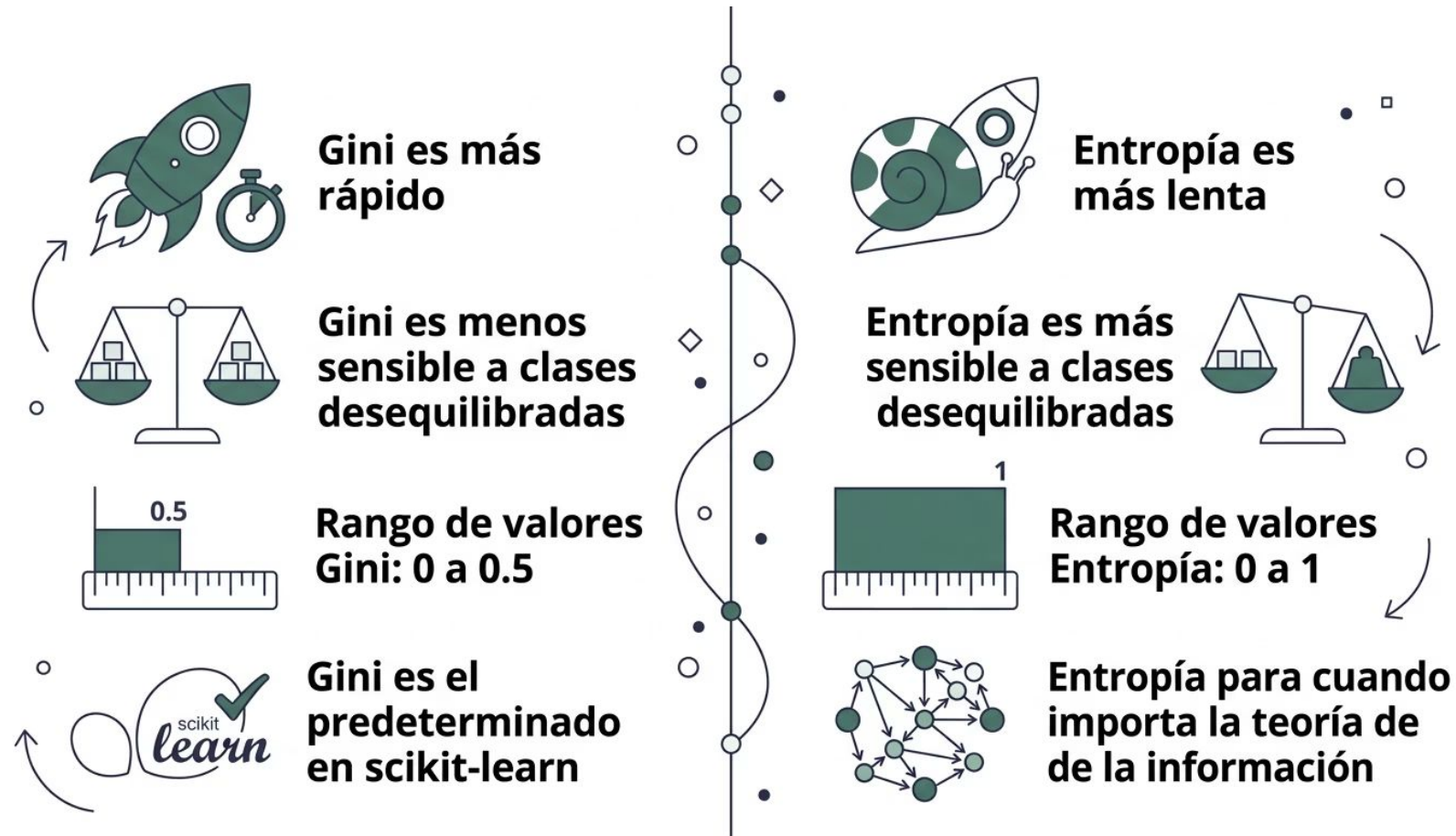
Mide la probabilidad de etiquetar incorrectamente un elemento elegido al azar. Vale **0** cuando el nodo es puro y alcanza su máximo cuando las clases están uniformemente distribuidas.

Entropía de Shannon

$$H = - \sum_k p_k \log_2 p_k$$

Cuantifica la *incertidumbre* de la distribución de clases. La **ganancia de información** es la reducción de entropía tras una partición. Más costosa computacionalmente, pero equivalente en resultados prácticos.

Comparación práctica: Gini vs. Entropía



En la práctica, ambos criterios producen árboles muy similares. La elección entre ellos rara vez tiene un impacto decisivo en el rendimiento final del modelo. **Gini** es el valor por defecto en `scikit-learn` por su menor coste computacional.

Profundidad y sobreajuste

Sin restricciones, un árbol de decisión crece hasta memorizar perfectamente el conjunto de entrenamiento: **cada hoja contiene exactamente una muestra**. Esto se denomina *sobreajuste* (*overfitting*): el modelo capta el ruido de los datos en lugar de sus patrones generales, y falla al generalizar a datos nuevos.

Profundidad 1–3

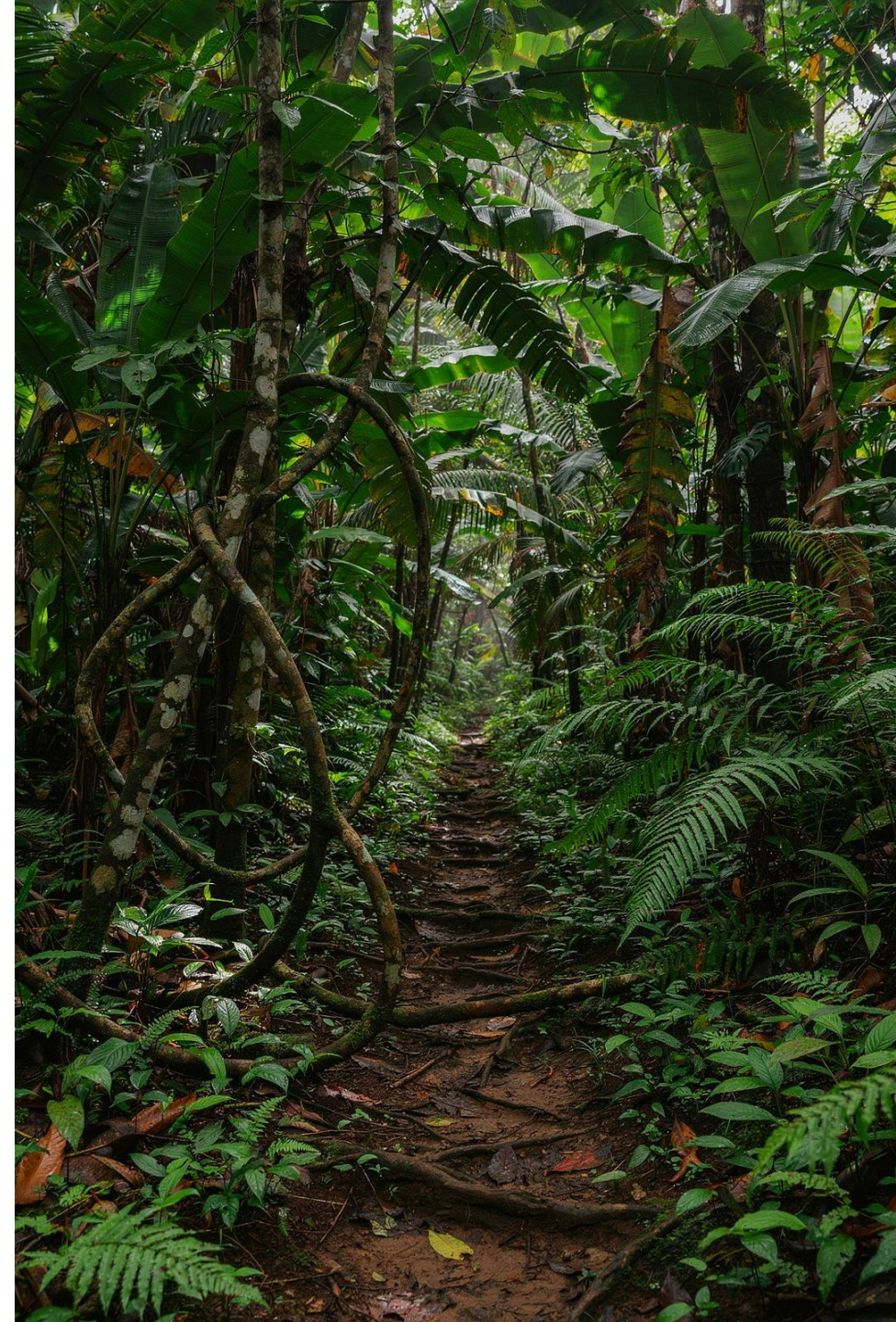
Modelo simple, posible subajuste.
Alta sesgo, baja varianza.

Profundidad 5–8

Balance razonable entre sesgo y
varianza para muchos problemas
reales.

Profundidad ilimitada

Memorización completa del entrenamiento. Baja sesgo, altísima varianza.



Parámetros de regularización

Para controlar la complejidad del árbol se actúa sobre varios hiperparámetros antes o durante el entrenamiento. Ajustarlos correctamente mediante **validación cruzada** es clave para un buen rendimiento.



max_depth

Profundidad máxima del árbol. Limita el número de niveles de preguntas y es el parámetro más intuitivo para controlar la complejidad.



min_samples_leaf

Número mínimo de muestras en cada hoja. Evita hojas con muy pocos ejemplos que podrían representar ruido estadístico.



min_samples_split

Número mínimo de muestras que debe tener un nodo para poder dividirse. Valores altos producen árboles más conservadores.



max_features

Número de atributos considerados en cada división. Reducirlo introduce aleatoriedad y es la base de los *Random Forests*.

Poda del árbol

La **poda** (*pruning*) es la técnica de reducir deliberadamente el tamaño del árbol para mejorar su capacidad de generalización. Existen dos enfoques principales:

Poda anticipada (pre-pruning)

Se detiene el crecimiento antes de que el árbol se desarrolle completamente, usando los hiperparámetros de regularización (`max_depth`, `min_samples_leaf`, etc.). Es computacionalmente eficiente.

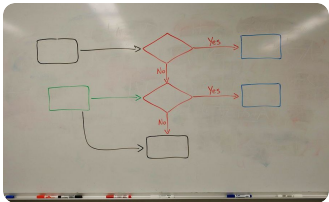
Poda posterior (post-pruning)

Se deja crecer el árbol completamente y después se eliminan ramas que no mejoran el rendimiento en validación. El método **Cost-Complexity Pruning** (`ccp_alpha` en scikit-learn) es el más utilizado en la práctica.

- ❏ Un árbol podado correctamente generaliza mejor que un árbol completo, aunque tenga ligeramente peor rendimiento en entrenamiento.

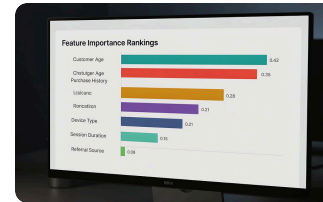
Interpretabilidad y visualización

Una de las mayores fortalezas de los árboles de decisión es su **transparencia**: el proceso de decisión puede representarse gráficamente y explicarse en lenguaje natural. Esto los convierte en la opción preferida cuando la interpretabilidad es un requisito del proyecto.



Visualización del árbol

Con `plot_tree` de scikit-learn o la librería `graphviz` se puede representar el árbol completo, mostrando en cada nodo el criterio de división, la impureza y el número de muestras.



Importancia de variables

El atributo `feature_importances_` indica cuánto contribuye cada variable a reducir la impureza total. Es una herramienta muy útil para la selección de características.

Árboles de regresión

El mismo algoritmo CART puede aplicarse a problemas de **predicción de valores continuos**. En lugar de impureza de clase, se minimiza el **error cuadrático medio** (MSE) en cada partición. Cada hoja predice el *valor medio* de las muestras que contiene.

Criterio de división en regresión

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \bar{y})^2$$

Se busca la partición que minimiza la suma ponderada del MSE de los dos nodos hijos. El resultado es una **función escalón constante a trozos**, lo que puede ser una limitación para relaciones muy suaves.

Características clave

- No asume linealidad en los datos.
- Capta automáticamente interacciones entre variables.
- Sensible a outliers en la variable objetivo.
- Predicciones constantes dentro de cada región.

Ventajas y límites

Ventajas

- **Interpretabilidad:** fáciles de explicar a no expertos.
- **Sin preprocesamiento:** no requieren normalización ni codificación de variables categóricas.
- **Tolerancia a outliers** y a valores faltantes.
- **Captura de no linealidades** e interacciones sin configuración explícita.
- **Velocidad** de entrenamiento e inferencia.

Límites

- **Alta varianza:** pequeños cambios en los datos pueden producir árboles muy diferentes.
- **Sobreajuste** si no se regulariza adecuadamente.
- **Fronteras de decisión ortogonales:** dificultad con relaciones diagonales.
- **Predicciones constantes** en regresión, sin extrapolación.
- Superados en rendimiento por **métodos ensemble** como Random Forests o Gradient Boosting.

Resumen y próximos pasos

Los árboles de decisión son un **punto de partida imprescindible** en aprendizaje automático: sencillos, interpretables y versátiles. Sus debilidades —alta varianza y tendencia al sobreajuste— se abordan directamente con las técnicas que estudiaremos a continuación.

- Domina los criterios Gini y entropía
Comprende cuándo y por qué se elige cada partición en el árbol.
- Practica la regularización
Experimenta con `max_depth` y `min_samples_leaf` en conjuntos reales usando scikit-learn.
- Avanza hacia los métodos ensemble
Random Forests y Gradient Boosting combinan múltiples árboles para superar sus limitaciones individuales.

